

Security Analysis of the "QuickShop POS" System

Course: Information Assurance and Security

System: QuickShop POS (Point-of-Sale) – A desktop and mobile-based retail management system.

Group Members:

John Doe
Lena Cruz
James Lee

Part I: Least Privilege

1. List of User Roles & Current Privileges

User Role	Expected Tasks	Actual Tasks
Cashier	Process sales, accept payments, print receipts, and view today's sales total. They should not alter prices.	Process sales, accept cash/card payments, print receipts, view today's sales total, and manually override the selling price of any item during checkout.
Inventory Manager	Add new products, update stock quantities, and generate reports. They should not delete categories—only archive them.	Add new products, update stock quantities after deliveries, generate inventory reports, and delete product categories entirely .
Admin	(Correct as-is) Manage user accounts, view logs, and perform backups. This is appropriately scoped.	Manage all user accounts (create/delete cashiers), view full sales history, access system logs, and perform database backups.

2. Identified Violation of Least Privilege

The most critical violation is the **Cashier's ability to manually override the selling price** of an item during checkout.

Currently, when a cashier scans a ₱500 item, a text box appears allowing them to change the price to any amount (e.g., ₱1) before finalizing the sale. The system does not require a manager's approval for this override.

3. Explanation of the Risk

If a dishonest cashier colludes with a friend (the "customer"), they could systematically reduce the prices of high-value items to nearly zero. The store would physically lose expensive inventory, but the financial records would show artificially low revenue. This fraud could go undetected for months, directly impacting the store's profitability. Furthermore, if a hacker compromises a single cashier's terminal, they could use this price-override feature to "purchase" items for free without needing to escalate their privileges to Admin level.

Part II: Defense in Depth

1. Critical Action Traced: Processing a Sale (Deducting Inventory & Finalizing Payment)

We traced the data flow of a single transaction—from the moment the cashier scans an item to the moment the database updates.

2. Current Layers of Protection

Layer	Description	Status
Layer 1: User Interface (POS App)	The POS application validates that the scanned barcode exists in the local product cache before adding it to the cart.	Implemented
Layer 2: Session Authentication	The server verifies that the Cashier has a valid, active JSON Web Token (JWT) for every API request.	Implemented
Layer 3: Database Permissions	The application connects to the database using a dedicated pos_app user. This user only has UPDATE/INSERT permissions on the sales and inventory tables—it explicitly <i>cannot</i> drop tables or modify user accounts.	Implemented

3. Identified Missing Layer

We completely lack **Server-Side Input Validation** for the final transaction total.

Currently, the POS frontend calculates the total price (including the cashier's overrides) and sends this final sum to the backend API (e.g., POST /api/sale with payload { "total": 150 }). The server **trusts** this client-sent total blindly. It does not independently fetch the original product prices from the database to verify the math.

4. Proposed New Layer

We propose adding a **Server-Side Business Rule Engine** as a new middle layer.

Before finalizing any sale, the backend must:

1. Ignore the total sent by the client.
2. Fetch the current official prices for all product IDs in the cart directly from the database.
3. Recalculate the total server-side.
4. Compare the server-calculated total with the client-sent total.
5. If they do not match within a tolerance of ₱0.01, the server immediately rejects the transaction with a generic "Invalid Request" error and logs the discrepancy for the Admin.

This layer ensures that even if the UI is hacked or a cashier abuses the price-override box, the database will refuse to process the fraudulent total.

Part III: Fail Secure

1. Critical Failure Point Identified: Central Database Server Goes Offline

Our POS system relies on a single MySQL database hosted on a local server. If this server crashes, loses network connectivity, or times out, the system cannot verify inventory or record sales.

2. The "Fail Open" Scenario

Currently, our system has a dangerous fallback behavior: if the database is unreachable, the POS terminal displays a pop-up: *"Database offline. Sale will be saved locally and synced later."*

It then allows the cashier to continue processing sales, temporarily saving transactions to a local JSON file on the cashier's PC. This is a **Fail Open** state because the system is "guessing" that it will be okay to process the sale later. It completely bypasses the inventory deduction check to keep the line moving.

3. The "Fail Secure" Redesign

We will redesign this behavior as follows:

- **User Experience:** If the database is unreachable, the system immediately halts all new sale transactions. The cashier sees a sterile message: *"System temporarily unavailable. Please contact the administrator or check network connection."*
- **Transaction Handling:** All current in-progress carts are cleared. No payment is captured.
- **Internal Logging:** The system secretly logs the exact timestamp, the cashier's User ID, and the specific database error (e.g., "Connection Timeout") to a secure, local audit file that only the Admin can read.

4. Justification of the Inconvenience

We acknowledge that blocking all sales during a database outage will frustrate customers during a busy lunch rush. However, this temporary inconvenience is vastly preferable to the risks of Failing

Open. If we allow offline sales, we face three major threats: (1) **Inventory Integrity**: We might sell items that were actually already purchased offline by another cashier, leading to negative stock counts. (2) **Data Loss**: The local JSON file could become corrupted, be overwritten, or the hard drive could fail before the sync occurs, resulting in permanent loss of revenue. (3) **Fraud**: A cashier could manually edit the local JSON file before the sync happens, fabricating lower totals. By failing secure, we guarantee that every single transaction is properly verified and recorded in the authoritative source of truth.

Part IV: Synthesis – Prioritizing The Next Security Fix

1. Chosen Priority

If we can only fix **one** major gap before our next release, we will implement the **Server-Side Business Rule Engine** (the missing Defense in Depth layer from Part 2).

2. Justification of Choice

This fix primarily supports the **Defense in Depth** principle, but it also reinforces **Least Privilege** (by rendering the cashier's price-override tool useless) and **Fail Secure** (by treating invalid totals as unrecoverable errors).

The Worst-Case Scenario if we ignore this:

A cashier with malicious intent (or a malware-infected POS terminal) could override a ₱10,000 television to ₱10. The frontend sends "₱10" to the backend, and the server—because it has no validation layer—accepts it, updates the database, and prints a receipt. The store loses ₱9,990 in a single transaction. Over a month, a single cashier could steal hundreds of thousands of pesos. The current audit logs would only show "Sale processed for ₱10," providing no evidence of foul play because the server blindly trusted the client.

How this fix makes our system resilient:

By forcing the server to recalculate the total independently, we eliminate the client's ability to lie about financial data. Even if the UI is completely hacked or the cashier tries to cheat, the backend acts as the "referee." This single change transforms our system from "trusting" to "verifying," which is the core of secure architecture. While other flaws exist (like the Inventory Manager deleting categories), financial theft is the most immediate and existential threat to a retail business, so it merits our immediate attention.